

Detail of Program Analysis

Ashish Mishra

Thanks to Rupesh Nasre @IITM for making slides public

Analysis Dimensions or Sensitivity

An analysis's precision and efficiency is guided by various design decisions.

- Flow-sensitivity
- Context-sensitivity
- Path-sensitivity
- Field-sensitivity



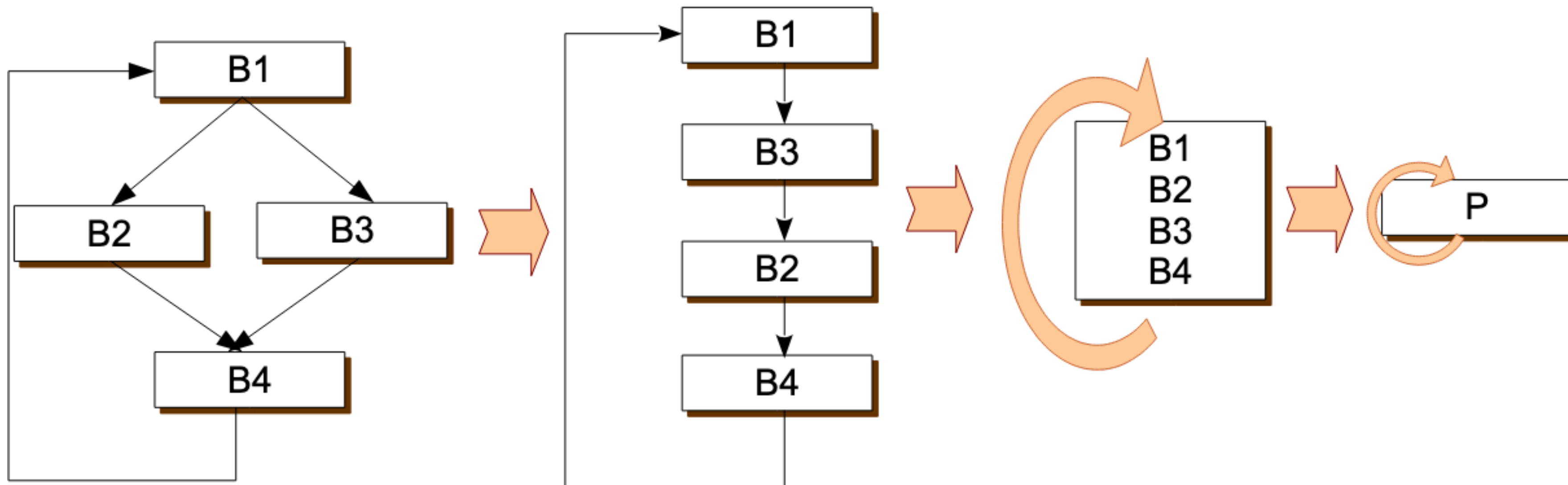
How many hands are required to know the time precisely?

Flow Sensitivity

L0: a = 0;
L1: a = 1;
L2: ...

Flow-sensitive solution: *at L1 a is 0, at L2 a is 1*
Flow-insensitive solution: *in the program a is in {0, 1}*

Flow-insensitive analyses ignore the control-flow in the program.



Flow Sensitivity

- Flow sensitive analysis
 - $x = 5$ so assertion will fail
- Flow insensitive analysis
 - $x = 5$ or 3

```
if (**) {  
    x = 3;  
    x = 5;  
    //assert ( x = 3)?  
}
```

Path Sensitivity

```
if (a == 0)
    b = 1;
else
    b = 2;
```

Path-sensitive solution:

b is 1 when a is 0, b is 2 when a is not 0

Path-insensitive solution:

b is in {1, 2} in the program

```
if (c1)
    while (c2) {
        if (c3)
            ...
        else
            for (; c4; )
                ...
    }
else
    ...
```

```
c1 and c2 and c3, ...
c1 and c2 and !c3 and c4, ...
c1 and c2 and !c3 and !c4, ...
c1 and !c2, ...
!c1 ...
...
```

Path Sensitivity

- Path sensitive analysis
 - No, as in both if branches, the condition is same and thus value of x in the second if used will be x=1
- Path insensitive analysis
 - Yes , as at the join, x is either 1 or 2.



```
x = 0
if (a > 0) {
    x = 1;
}else {
    x = 2;
}
if (a > 0) { x = x + 3}
// Can x be 5 here
```

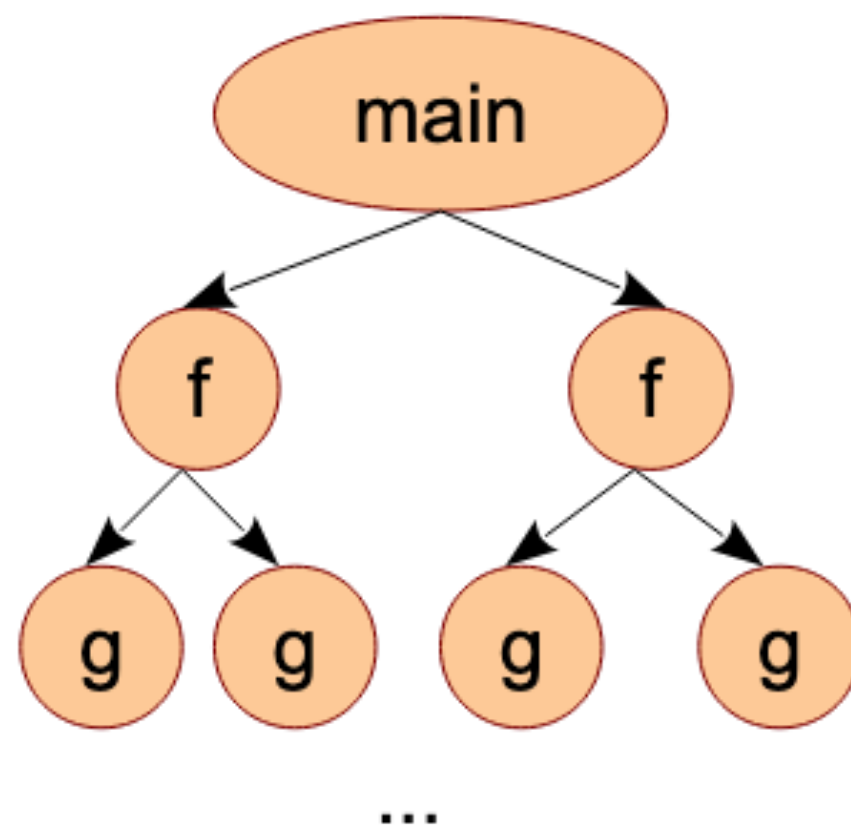

Context Sensitivity

```
main() {  
  L0: fun(0);  
  L1: fun(1);  
}
```

```
fun(int x) {  
  y = x;  
}
```

Context-sensitive solution:
y is 0 along L0, y is 1 along L1

Context-insensitive solution:
y is in {0, 1} in the program



Exponential
Number of
contexts

Along main-f1-g1, ...
Along main-f1-g2, ...
Along main-f2-g1, ...
Along main-f2-g2, ...

Exponential time requirement

Exponential storage requirement

Path Sensitivity

- Context sensitive analysis
 - No,
 - first context $n=0$ and $g = 3$
 - second context $n = 3$ and $g = 5$
- Context insensitive analysis
 - Merging all context (call-sites) into one blob
 - g 's call site , $n= 0$ or 3 , and $y = 3$ or 5 .
 - Yes.

```
n = 0
function f (x){
  if (x) then {
    | | g (3)
  }
  else {
    | | n = 3
    | | g (5)
  }
}
```

```
function g (y){
  // Can y be equal to n?
  ....
}
```


Field Sensitivity

```
struct T s;
```

```
s.a = 0;
```

```
s.b = 1;
```

Field-sensitive solution:

s.a is 0, s.b is 1

Field-insensitive solution:

s is in {0, 1}

Aggregates are collapsed into a single variable.
e.g., arrays, structures, unions.

This reduces the number of variables tracked during the analysis and reduces precision.

Example

- Find the values of variables in
 - context + flow-sensitive analysis
 - interprocedural context-insensitive but flow-sensitive analysis
 - intraprocedural flow-insensitive analysis

```
int g = 0;
void fun(int n) {
    g = n;
}
void main() {
    int a = 1;
    a = 2;
    fun(a); // L1
    a = 3;
    fun(a); // L2
}
```

Back to Ptr Analysis Again

Anderson's Analysis

- Inclusion-based / subset-based / constraint-based analysis
- Flow-insensitive analysis

For a statement $p = q$,

create a constraint $\text{ptsto}(p) \supseteq \text{ptsto}(q)$

where p is of the form $*a$, a , and q is of the form $*a$, a , $\&a$.

Solving these inclusion constraints results into the points-to solution.

Anderson's Analysis

Program

```
a = &x;  
b = &y;  
p = &a;  
c = b;  
*p = c;
```

Constraints

```
ptsto(a)  $\supseteq$  {x}  
ptsto(b)  $\supseteq$  {y}  
ptsto(p)  $\supseteq$  {a}  
ptsto(c)  $\supseteq$  ptsto(b)  
ptsto(*p)  $\supseteq$  ptsto(c)
```

fixed-point

Pointers	Iteration 0	Iteration 1	Iteration 2
a	{ }	{x, y}	
b	{ }	{y}	
c	{ }	{y}	
p	{ }	{a}	
x	{ }		
y	{ }		

Imprecision

Anderson's Analysis modified

Program

```
a = &x;  
b = &y;  
p = &a;  
*p = c;  
c = b;
```

Constraints

```
ptsto(a)  $\supseteq$  {x}  
ptsto(b)  $\supseteq$  {y}  
ptsto(p)  $\supseteq$  {a}  
ptsto(*p)  $\supseteq$  ptsto(c)  
ptsto(c)  $\supseteq$  ptsto(b)
```

Order does not matter
for correctness,
but it does matter
for efficiency.

fixed-point

Pointers	Iteration 0	Iteration 1	Iteration 2	Iteration 3
a	{ }	{x}	{x, y}	
b	{ }	{y}		
c	{ }	{y}		
p	{ }	{a}		
x	{ }			
y	{ }			

Anderson's Analysis Example

Program

```
*p = c;  
b = &y;  
b = *p;  
p = &a;  
a = &x;  
*p = c;  
c = p;  
c = &z;
```

Constraints

```
ptsto(*p)  $\supseteq$  ptsto(c)  
ptsto(b)  $\supseteq$  {y}  
ptsto(b)  $\supseteq$  ptsto(*p)  
ptsto(p)  $\supseteq$  {a}  
ptsto(a)  $\supseteq$  {x}  
ptsto(*p)  $\supseteq$  ptsto(c)  
ptsto(c)  $\supseteq$  ptsto(p)  
ptsto(c)  $\supseteq$  {z}
```

fixed-point

Pointers	Iteration 0	Iteration 1	Iteration 2	Iteration 3
a	{ }	{x}	{a, x, z}	
b	{ }	{y}	{a, x, y, z}	
c	{ }	{a, z}	{a, z}	
p	{ }	{a}	{a}	
x	{ }			
y	{ }			
z				

Steensgaard's Analysis

For a statement $p = q$, merge the points-to sets of p and q .

In subset terms, $\text{ptsto}(p) \supseteq \text{ptsto}(q)$ **and** $\text{ptsto}(q) \supseteq \text{ptsto}(p)$ with a single representative element.

Less precise but more efficient than Anderson's

Both are less precise but more efficient than the initial precise algorithm A

Steenggaard's Analysis Example

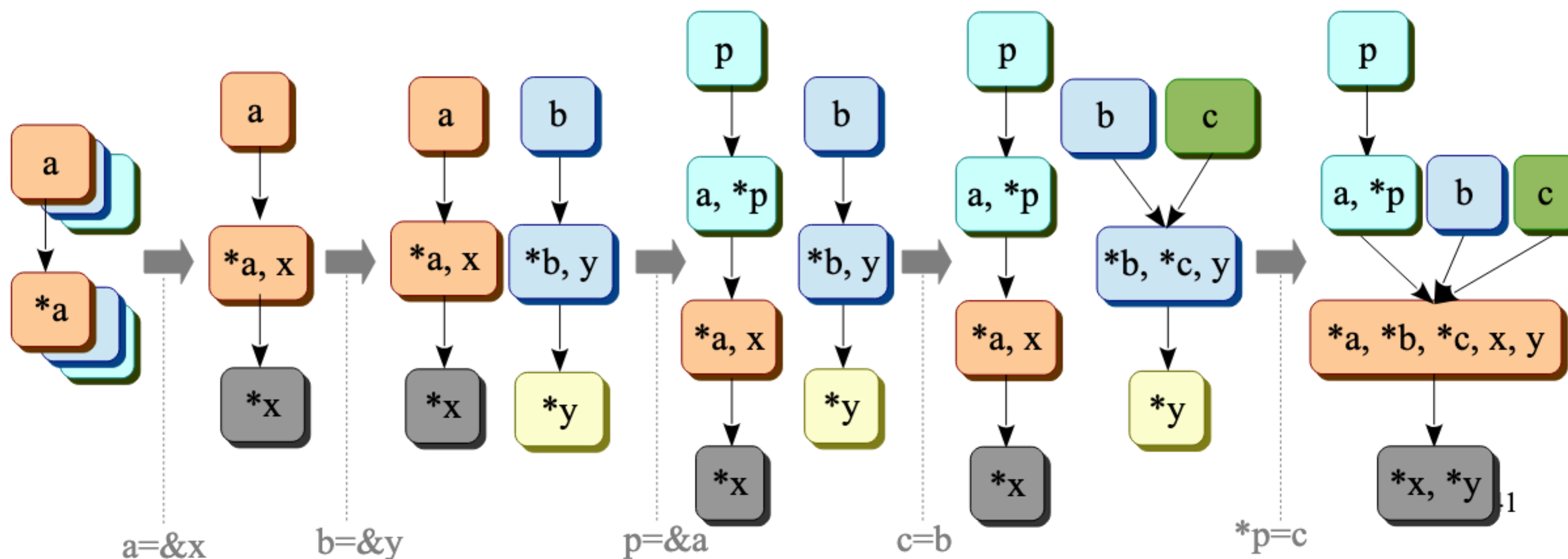
Program	Andersen's	Steensgaard's
<code>a = &x;</code> <code>b = &y;</code> <code>p = &a;</code> <code>c = b;</code> <code>*p = c;</code>	$a \rightarrow \{x, y\}$ $b \rightarrow \{y\}$ $c \rightarrow \{y\}$ $p \rightarrow \{a\}$	$a \rightarrow \{x, y\}$ $b \rightarrow \{x, y\}$ $c \rightarrow \{x, y\}$ $p \rightarrow \{a\}$

Pointers	Iteration 0	Iteration 1
a	$\{*a\}$	$\{*a, *b, *c, x, y\}$
b	$\{*b\}$	$\{*a, *b, *c, x, y\}$
c	$\{*c\}$	$\{*a, *b, *c, x, y\}$
p	$\{*p\}$	$\{*p, a\}$
x	$\{*x\}$	
y	$\{*y\}$	

Only one iteration

Steengaard's Analysis Under the hood

Program	Andersen's	Steensgaard's
$a = \&x;$ $b = \&y;$ $p = \&a;$ $c = b;$ $*p = c;$	$a \rightarrow \{x, y\}$ $b \rightarrow \{y\}$ $c \rightarrow \{y\}$ $p \rightarrow \{a\}$	$a \rightarrow \{x, y\}$ $b \rightarrow \{x, y\}$ $c \rightarrow \{x, y\}$ $p \rightarrow \{a\}$



Other resources

Precision		← Precision	
Precision ↑		Context-Sensitive	Context-Insensitive
	Flow-Sensitive	Landi, Ryder 92 Choi et al. 93 Emami et al. 94 Reps et al. 95 Hind et al. 99 Kahlon 08	Zheng 98 Hardekopf, Lin 09
	Flow-insensitive	Liang, Harrold 99 Whaley, Lam 04 Zhu, Calman 04 Lattner et al. 07	Andersen 94 Steensgaard 96 Shapiro, Horwitz 97 Fahndrich et al. 98 Das 00 Rountev, Chandra 00 Berndl et al. 03 Hardekopf, Lin 07 Pereira, Berlin 09 Mendez-Lojo 10
	Surveys	Hind, Pioli 00 Qiang, Wu 06	